

Google Vertex AI Model Garden ile Fine-Tuning

Temelden Uygulamaya Rehber

Güncel: Temmuz 2026

Bölüm 1 — Model Garden nedir, Hugging Face'ten farkı ne?

Model Garden, Vertex AI (yeni adıyla "Agent Platform") içinde yüzlerce hazır modelin (Gemini, Gemma, Llama, Claude, imaj/embedding modelleri) bulunduğu katalog + bunları **tek tıkla tune/deploy** etme arayüzüdür.

Hugging Face'te altyapıyı siz yönetirsiniz (GPU kiralı, TRL kodu yaz). Model Garden'da iki felsefe vardır:

Kriter	Managed Tuning (Gemini)	Open-model Tuning (Gemma/Llama)
Altyapı	Tamamen Google yönetir, GPU görmezsiniz	Sizin GPU/TPU'nuzda pipeline çalışır
Yöntem	LoRA (arka planda, <code>adapter_size</code> ile)	LoRA/QLoRA (notebook'ta siz kurarsınız)
Faturalama	Training token başına	GPU/TPU-saat başına
Kod	Birkaç satır SDK / no-code konsol	Notebook (vertex-ai-samples)
Ne zaman	Gemini kalitesi + sıfır ops istiyorsanız	Ağırlıklar sizde kalacaksa, açık model şartsa

Temel kavramlar (token, epoch, LoRA, overfitting, QLoRA, SFT → DPO) Hugging Face rehberindekiyle birebir aynıdır — fark **operasyon modelinde**, teoride değildir. Bu rehber Vertex'e özgü akışa odaklanır.

Bölüm 2 — Ön Hazırlık (iki yol için ortak)

- GCP projesi + faturalama** açık olmalı.
- API'leri etkinleştir:** Vertex AI API, Cloud Storage.
- Cloud Storage bucket:** Eğitim verisi (JSONL) ve çıktılar buraya. Bölge modelle aynı olmalı (ör. `us-central1`).
- IAM rolleri:** `Vertex AI User` + `Storage Admin` (veya daha dar yetki).
- SDK kurulumu:**

```
pip install --upgrade google-genai # Gemini managed tuning (güncel SDK)
pip install --upgrade google-cloud-aiplatform # açık model tuning için ek
```

Bölüm 3 — Managed Tuning (Gemini): Konsol Akışı

Model Garden ekranından:

- Model kartını aç** (ör. Gemini 2.5 Flash) → **"Fine tune"** → **"Managed tuning"**. (veya sol menü → *Tuning* → "Create tuned model")
- Tuning method:** Supervised fine-tuning.
- Base model** seç.
- Training dataset:** GCS'deki JSONL'i göster (format aşağıda).
- Hiperparametreler:** - *Epochs* — genelde 1–5 (Google ~100+ örnekle başlamayı önerir). - *Adapter size* — LoRA rank'i (1/2/4/8/16). Küçük veri → küçük değer. - *Learning rate multiplier* — 1.0 varsayılan; overfit'te düşür.
- Validation dataset** (opsiyonel ama önerilir) → tuning sırasında metrik grafiği.
- Start tuning** → iş "Managed tuning" sekmesinde görünür. Bitince adapter **Model Registry**'ye kaydolur.
- Deploy** → bir **Endpoint** oluşur (Gemini + sizin adapter'ınız). Endpoint **açık kaldığı sürece saatlik ücretlenir** — kullanmıyorsanız undeploy edin.

Veri formatı (JSONL — her satır bir örnek)

```
{ "systeminstruction": { "role": "system", "parts": [{ "text": "Sen bir hukuk asistanisin." }] }, "contents": [{ "role": "user", "parts": [{ "text": "Bu sözleşmeyi"
```

- `role` değerleri `user` ve `model` (Hugging Face'teki `assistant` değil — dikkat).
- Multi-turn destekler; sistem talimatı opsiyoneldir.

SDK ile (konsol yerine kod)

```
from google import genai
from google.genai.types import CreateTuningJobConfig, TuningDataset

client = genai.Client(vertexai=True, project="<PROJE_ID>",
                      location="us-central1")

job = client.tunings.tune(
    base_model="gemini-2.5-flash",
    training_dataset=TuningDataset(gcs_uri="gs://<bucket>/train.jsonl"),
    config=CreateTuningJobConfig(
        epoch_count=3,
        adapter_size=4,  # LoRA rank
        learning_rate_multiplier=1.0,
        tuned_model_display_name="hukuk-ozet-v1",
        validation_dataset=TuningDataset(gcs_uri="gs://<bucket>/val.jsonl"),
    ),
)
print(job.name)  # tuning job kaynagi
# Bitince: client.models.generate_content(model=job.tuned_model.endpoint, ...)
```

API doğrulama notu: `google-genai` `tunings.tune` imzası güncel yoldur; SDK sürümü hızlı değiştiğinden çalıştırmadan önce `help(client.tunings)` ile alan adlarını doğrulayın (argüman isimleri sürümler arası değişebiliyor).

Bölüm 4 — Open-model Tuning (Gemma / Llama): Notebook Akışı

Gemini yerine **açık ağırlıklı** model tune etmek (ve ağırlıkları elde tutmak) istiyorsanız:

- Model Garden'da modeli aç (ör. **Gemma 3** veya **Llama 3.1**) → "Fine tune" genelde bir **Colab Enterprise / Workbench notebook**'u açar (`GoogleCloudPlatform/vertex-ai-samples` deposundan).
- Notebook, arka planda bir **Vertex AI custom/pipeline job** başlatır: seçtiğiniz GPU/TPU'da (ör. A100/H100/TPU v5e) LoRA/QLoRA ile eğitim.
- Çıktı adapter/model GCS + Model Registry'ye yazılır → Endpoint'e deploy.

Bu yol teknik olarak **Hugging Face TRL akışının Google altyapısındaki halidir**; farkı: GPU'yu Vertex sağlar, işi pipeline yönetir, faturalama GPU-saat üzerinden gider.

Bölüm 5 — Ücretler (Temmuz 2026, göstergesel)

Fiyatlar oynaktır; işlem öncesi resmî Vertex AI fiyat sayfasından doğrulayın.

5.1 Gemini Managed Tuning

- Eğitim:** *training token* başına — pratikte **dataset token × epoch sayısı**. Örn. Gemini 2.0/2.5 Flash eğitimi ~\$3 / 1M training token civarı.
- Inference (tuned model):** Flash sınıfında genelde **taban modelle aynı fiyat, tuning primi yok** — yüksek hacimde büyük avantaj.
- Endpoint hosting:** Model deploy'da kaldığı sürece altyapı ücreti.
- Maliyet tahmini:** Google'ın token-sayım notebook'u ile eğitim öncesi hesaplanabilir.

5.2 Open-model (Gemma/Llama) Tuning

- Eğitim:** GPU/TPU-saat başına (Vertex custom job fiyatı). Küçük Gemma LoRA işi birkaç dolar; büyük Llama işi onlarca-yüzlerce dolar.
- Inference:** Vertex, açık modeli host etmek için **yönetim primi** ekler (ör. Llama 3.3 70B Vertex'te ~\$1.36/1M token — salt sağlayıcılara göre daha pahalı).
- Endpoint:** Deploy açık kaldıkça GPU saati ücretlenir → **kullanmadığında undeploy et** (en sık maliyet tuzağı).

5.3 Ücretsiz / Düşük Maliyet

- \$300 GCP free trial** (yeni hesaplar) — tuning denemeleri için kullanılabilir.
- Küçük dataset + az epoch + Flash sınıfı model → Gemini managed tuning çok ucuz.
- Salt öğrenme amacıyla: **Gemma'yı Hugging Face rehberindeki gibi Colab/Kaggle free'de tune et**, Vertex'i sadece production deploy için kullan.

5.4 Maliyet Disiplini (kritik)

- Endpoint'i boşa bırakma (saatlik yanar).
- Bölgeyi veri + model + bucket için aynı tut (cross-region ağ ücreti).
- Epoch'u minimumda tut; val loss ile erken dur.
- Eğitim öncesi token/maliyet tahmini yap.

Bölüm 6 — Hugging Face vs Vertex Model Garden: Hangisi Ne Zaman?

Ağırlık sizin olacak, taşınabilirlik/gizlilik şart	-> HF (TRL) veya Vertex open-model
En düşük compute maliyeti, tam kontrol	-> HF + Vast.ai/RunPod
Sıfır ops, Gemini kalitesi, hızlı production	-> Vertex Managed Tuning
Zaten GCP ekosistemindesiniz (BigQuery, IAM, VPC)	-> Vertex
Sadece öğrenme / bütçe \$0	-> HF + Kaggle/Colab free
Kurumsal uyumluluk, endpoint SLA, ölçekli serving	-> Vertex Endpoint

Özet: Model Garden = "yönetilen kolaylık + GCP entegrasyonu, karşılığında premium". Hugging Face = "en ucuz + tam kontrol, karşılığında ops yükü". Gemini'ye özel bir davranış istiyorsanız Vertex tek yoldur; açık model + taşınabilirlik istiyorsanız her ikisi de olur.

Kaynaklar

- About supervised fine-tuning for Gemini — Google Cloud Docs
- Supervised & distillation tuning for open models — Google Cloud Docs
- Model Garden fine-tuning tutorial notebook — GitHub (GoogleCloudPlatform/vertex-ai-samples)
- Gemma fine-tuning on Vertex — GitHub notebook
- Agent Platform / Vertex AI Pricing — cloud.google.com/vertex-ai/generative-ai/pricing
- Fine-tune Gemini on Vertex AI — Google Codelabs

Fiyat ve SDK bilgileri Temmuz 2026 itibarıyladır ve değişebilir.